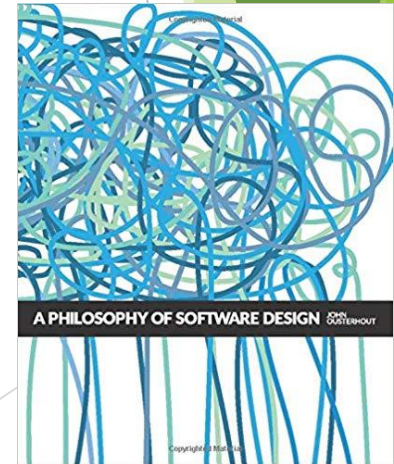


Princípios de Projeto

"O problema mais fundamental em Ciência da Computação é a tarefa de decomposição de problemas: como dividir um problema complexo em partes que possam ser resolvidas de forma independente" -- John Ousterhout



Definição de Projeto de Software

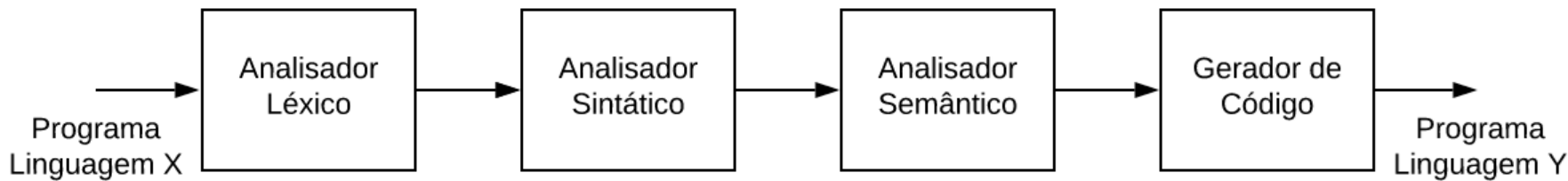
- ↴ Frase de Ousterhout é uma excelente definição
- ↴ Projeto:
 - ↵ Quebrar um "problema grande" em partes menores
 - ↵ Resolução (ou implementação) das partes menores resolvem (ou implementam) o "problema grande"

Project vs Design

- ↴ Em Inglês, temos duas palavras:
 - ↵ Project: esforço colaborativo para resolver problemas
 - ↵ Design: desenho ou proposta de uma solução
- ↴ Em Português, temos uma única palavra: projeto
- ↴ Nesta aula, **projeto = design**

Projetar = Quebrar em partes menores

↴ Exemplo: compilador



Módulos

- ↴ Em Engenharia de Software:
 - ↵ Partes menores = módulos (pacotes, componentes, classes, etc)

O que vamos estudar? (parte 1)

↴ Propriedades de "bons projetos" de software

- ↵ Integridade Conceitual
- ↵ Ocultamento de Informação
- ↵ Coesão
- ↵ Acoplamento

O que vamos estudar? (parte 2)

- ↴ Princípios (ou diretrizes) para projeto de módulos com as propriedades que estudamos antes:
 - ↵ Responsabilidade Única
 - ↵ Segregação de Interfaces
 - ↵ Prefira Interfaces a Classes
 - ↵ Aberto/Fechado
 - ↵ Demeter
 - ↵ Substituição de Liskov

Propriedades de Projeto

Integridade Conceitual

Integridade Conceitual = coerência e padronização de funcionalidades, projeto e implementação



Exemplo



Contra-Exemplo

Integridade Conceitual

↴ Decisões de um sistema devem ser coerentes

↴ Sejam elas sobre:

↵ Funcionalidades

↵ Interface com usuário

↵ Projeto

↵ Implementação

↵ etc

Exemplo (referente a interface com o usuário)

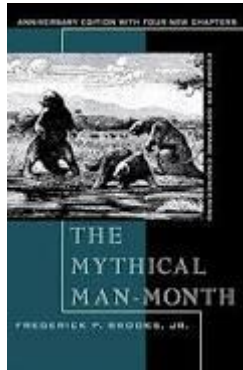
- ↴ Botão "sair" é idêntico em todas as telas
- ↴ Se um sistema usa tabelas para apresentar resultados, todas as tabelas têm o mesmo *layout*
- ↴ Todos os resultados são mostrados com 2 casas decimais

Integridade Conceitual vale também para o
projeto e código de um sistema

Exemplos (em nível de projeto/código)

- ↴ Todas as variáveis seguem o mesmo padrão de nomes
 - ↵ contra-exemplo: `nota_total` vs `notaMedia`
- ↴ Todas as páginas usam o mesmo framework (mesma versão)
- ↴ Se um problema é resolvido com uma estrutura de dados X, todos os problemas parecidos também usam X

Integridade conceitual é a consideração mais importante no projeto de sistemas -- Fred Brooks



Motivo: integridade conceitual facilita uso e entendimento de um sistema

Ocultamento de Informação (Information Hiding)

Origem do conceito (David Parnas, 1972)

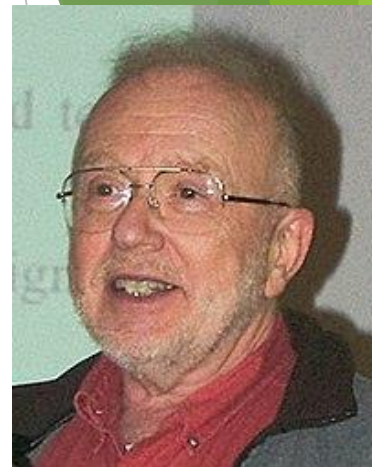
On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time. The effectiveness of a “modularization” is dependent upon the criteria used in dividing the system into modules. A system design problem is presented and

Introduction

A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:¹



```
import java.util.Hashtable;

public class Estacionamento {

    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

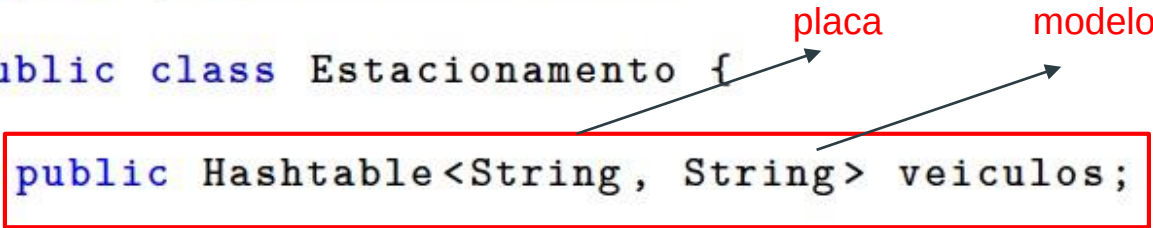
    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```

```
import java.util.Hashtable;

public class Estacionamento {
    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```



```
import java.util.Hashtable;

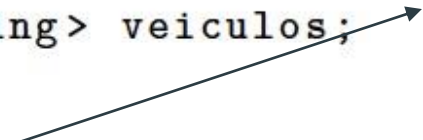
public class Estacionamento {

    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```

Construtora, cria a
Hashtable



```
import java.util.Hashtable;

public class Estacionamento {

    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```

Problema: clientes precisam manipular uma estrutura de dados interna da classe, para estacionar um veículo

Problema

- ↴ Classes precisam de um pouco de "privacidade"
- ↴ Até para evoluir de forma independente dos clientes
- ↴ Código anterior: clientes manipulam a hashtable
- ↴ Comparação: clientes não podem entrar na cabine do estacionamento e eles mesmo anotar os dados do seu carro no "livro" do estacionamento

Agora uma versão com ocultamento de
informação

1

```
import java.util.Hashtable;

public class Estacionamento {

    private Hashtable<String,String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public void estaciona(String placa, String veiculo) {
        veiculos.put(placa, veiculo);
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.estaciona("TCP-7030", "Uno");
        e.estaciona("BNF-4501", "Gol");
        e.estaciona("JKL-3481", "Corsa");
    }
}
```

2

```
import java.util.Hashtable;

public class Estacionamento {

    private Hashtable<String,String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public void estaciona(String placa, String veiculo) {
        veiculos.put(placa, veiculo);
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.estaciona("TCP-7030", "Uno");
        e.estaciona("BNF-4501", "Gol");
        e.estaciona("JKL-3481", "Corsa");
    }
}
```

3

```
import java.util.Hashtable;

public class Estacionamento {

    private Hashtable<String,String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public void estaciona(String placa, String veiculo) {
        veiculos.put(placa, veiculo);
    }

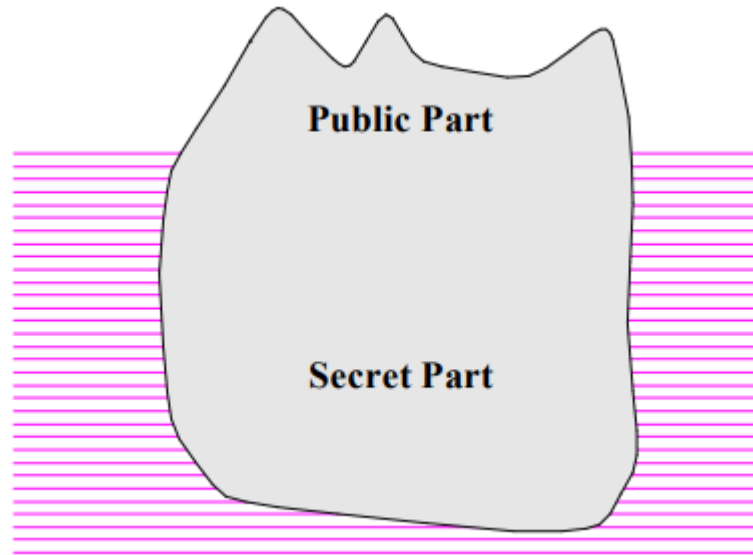
    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.estaciona("TCP-7030", "Uno");
        e.estaciona("BNF-4501", "Gol");
        e.estaciona("JKL-3481", "Corsa");
    }
}
```

Resultado: classe Estacionamento fica livre para alterar a sua estrutura de dados interna

Ocultamento de Informação

- ↴ Classes devem ocultar detalhes internos de sua implementação (usando modificador **private**)
- ↴ Principalmente aqueles sujeitos a mudanças
- ↴ Adicionalmente, interface da classe deve ser estável
- ↴ **Interface** = conjunto de métodos públicos de uma classe

Bom módulos são semelhantes a icebergs
(pequena parte pública e visível; grande parte submersa e privativa)



Fonte: Bertrand Meyer, Object-oriented software construction, 1997 (pág. 51)

Coesão

Coesão

- ↴ Uma classe deve ter uma única função, isto é, oferecer um único serviço
- ↴ Vale também para outras unidades: funções, métodos, pacotes, etc.

Contra-exemplo 1

```
float sin_or_cos(double x, int op) {  
    if (op == 1)  
        "calcula e retorna seno de x"  
    else  
        "calcula e retorna cosseno de x"  
}
```

Contra-exemplo 1

```
float sin_or_cos(double x, int op) {  
    if (op == 1)  
        "calcula e retorna seno de x"  
    else  
        "calcula e retorna cosseno de x"  
}
```

Deveria ser quebrada em duas funções: sin e cos

Contra-exemplo 2

```
class Estacionamento {  
    ...  
    private String nome_gerente;  
    private String fone_gerente;  
    private String cpf_gerente;  
    private String endereco_gerente;  
    ...  
}
```

Contra-exemplo 2

```
class Estacionamento {  
    ...  
    private String nome_gerente;  
    private String fone_gerente;  
    private String cpf_gerente;  
    private String endereco_gerente;  
    ...  
}
```

Deveria ser quebrada em duas classes:
Estacionamento e Gerente

Exemplo

```
class Stack<T> {  
    boolean empty() { ... }  
    T pop() { ... }  
    push (T) { ... }  
    int size() { ... }  
}
```

Todos esses métodos manipulam os elementos da Pilha

Acoplamento

Acoplamento

- ↴ Nenhuma classe é uma ilha ...
- ↴ Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)
- ↴ A questão principal é a **qualidade** desse acoplamento
- ↴ Dois tipos:
 - ↵ Acoplamento aceitável ("bom")
 - ↵ Acoplamento ruim

Acoplamento Aceitável

- ↴ Classe A depende de uma classe B:
 - ↵ Mas a classe B possui uma interface estável
 - ↵ Classe A somente chama métodos da interface de B


```
import java.util.Hashtable;
```

```
public class Estacionamento {
```

Classe Estacionamento depende (está acoplada) à classe Hashtable, mas esse acoplamento é aceitável

```
    private Hashtable<String,String> veiculos;
```

```
    public Estacionamento() {  
        veiculos = new Hashtable<String, String>();  
    }
```

```
    public void estaciona(String veiculo, String placa) {  
        veiculos.put(veiculo, placa);  
    }
```

```
    public static void main(String[] args) {  
        Estacionamento e = new Estacionamento();  
        e.estaciona("TCP-7030", "Uno");  
        e.estaciona("BNF-4501", "Gol");  
        e.estaciona("JKL-3481", "Corsa");  
    }
```

```
}
```

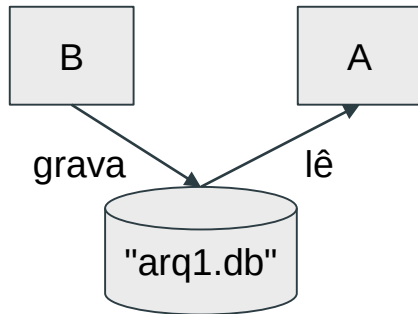
Acoplamento Ruim

- ↴ Classe A depende de uma classe B:
 - a. Mas interface da classe B é instável
 - b. Ou então a dependência não ocorre via interface de B

Como uma classe A pode depender de uma classe B sem ser via a interface de B?

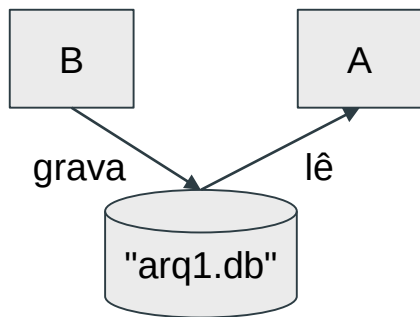
```
class A {  
  
    private void f() {  
        int total;  
        ...  
        File f = File.open("arq1.db");  
        total = f.readInt();  
        ...  
    }  
}
```

```
class B {  
    private void g() {  
        int total;  
        // computa valor de total  
        File f = File.open("arq1.db");  
        f.writeInt(total);  
        ...  
        f.close();  
    }  
}
```



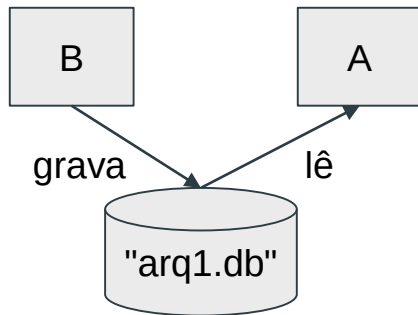
```
class A {  
  
    private void f() {  
        int total;  
        ...  
        File f = File.open("arq1.db");  
        total = f.readInt();  
        ...  
    }  
  
}
```

```
class B {  
    private void g() {  
        int total;  
        // computa valor de total  
        File f = File.open("arq1.db");  
        f.writeInt(total);  
        ...  
        f.close();  
    }  
}
```



Problema

- ↴ Classe B não sabe que a classe A é sua "cliente"
- ↴ Logo, B pode mudar o formato do arquivo ou mesmo deixar de salvar o dado que é lido por A



Tornando acoplamento ruim em bom

```
class B {  
  
    int total;  
  
    public int getTotal() {  
        return total;  
    }  
  
    private void g() {  
        // computa valor de total  
        File f = File.open("arq1");  
        f.writeInt(total);  
        ...  
    }  
}
```

```
class A {  
  
    private void f(B b) {  
        int total;  
        total = b.getTotal();  
        ...  
    }  
}
```

Tornando acoplamento ruim em bom

```
class B {  
  
    int total;  
  
    public int getTotal() {  
        return total;  
    }  
  
    private void g() {  
        // computa valor de total  
        File f = File.open("arq1");  
        f.writeInt(total);  
        ...  
    }  
}
```

```
class A {  
  
    private void f(B b) {  
        int total;  
        total = b.getTotal();  
        ...  
    }  
}
```


Resumo

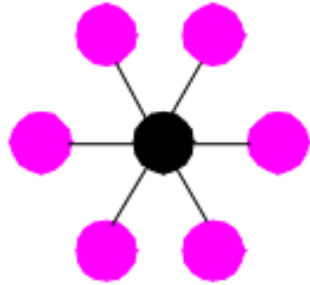
- ↵ Acoplamento estático (ou estrutural):
 - ↵ Classe A depende de uma classe B
 - ↵ No código de A, existe uma referência explícita para B
 - ↵ Pode ser um acoplamento bom ou ruim
- ↵ Acoplamento evolutivo (ou lógico):
 - ↵ No código de A, não existe uma referência para B
 - ↵ Porém, mudanças em B podem impactar a classe A
 - ↵ Tipo de acoplamento sempre ruim

Recomendação comum em projeto de software:

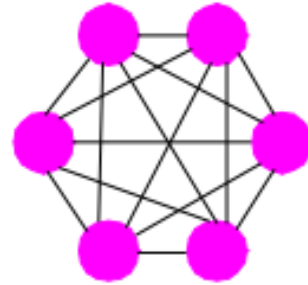
Maximize a coesão, minimize o acoplamento

Mas cuidado: minimize (ou elimine)
principalmente o acoplamento ruim

Genericamente falando, qual dos seguintes projetos é melhor?
Justifique (nodos = módulos; arestas = dependências)



(A)



(B)

Fonte: Bertrand Meyer, Object-oriented software construction, 1997 (pág. 47)

Referências

- ↴ PRESSMAN, R. S.. Engenharia de Software. Makron Books. 1995.
- ↴ Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020 (<https://engsoftmoderna.info/>)